

國立成功大學
測量及空間資訊學系
碩士論文

天際線查詢應用於具時間限制之多需求路徑規劃
Skyline Query for Multi-Request Route Planning
with Time Constraint

研 究 生：陳良允

指導教授：呂學展

中華民國一〇九年七月

中文摘要

適地性服務 (Location-Based Service, LBS) 已成為日常生活中不可或缺的一部份，其中一個重要的 LBS 為路徑規劃。有別於傳統路徑規劃問題，多需求路徑規劃 (Multi-Request Route Planning, MRRP) 考慮一個興趣點 (Point of Interest, POI) 可以包含多種使用者需要的服務，如吃飯，領錢等。然而 MRRP 沒有考慮查詢時間，無法反應使用者需求的急迫性與店家營業時間等問題。本研究認為 MRRP 除了要考慮 POI 有多種服務的特性外，也應該考慮查詢時間特性。由於加入時間維度，MRRP 的目標是將從單一維度的最短路徑搜尋調整為多重維度的最佳路徑搜尋，在決策支援系統中，當無法定義多重維度好壞時，天際線查詢適合用於搜尋不同維度上不被支配的解。因此本論文提出一個具時間限制之多需求路徑規劃 (MRRP with Time Urgency, MRRP-TU) 問題，為了有效地搜尋具時間限制之多需求路徑，我們設計了具有天際線查詢之 MRRP-TU 隨時演算法 (Anytime Algorithm)，其中結合優先權佇列、天際線修剪策略與潛在成本估計，透過最短路徑與延遲時間，使用者可以在多條路徑中去選擇自己偏好的路徑。在實驗中，我們使用了台南的 POI 資料，在最佳解演算法中，我們的方法可以大幅的增加我們的查詢速度，也顯示出能夠快速返回解的能力，在近似解的實驗，也顯示我們提出的方法可以在短時間內返回高品質的解。

關鍵詞：多需求路線規劃、時間限制、天際線查詢、適地性服務。

Abstract

Location-Based Service (LBS) has become an integral part of daily life. An important part of LBS is route planning. Unlike classical route planning problems, Multi-Request Route Planning (MRRP) Consider a Point of Interest (POI), MRRP can include various services that users need, such as eating meals, getting money, etc. However, MRRP does not consider the query time, which does not respond to the urgency of users' requests or the opening hours of stores. In this study, we consider that MRRP should consider POI with multiple services and query time. With time dimension, MRRP aims to move from single-dimension shortest path searches to multi-dimension optimal paths searches. When there is no defined multi-dimensional advantage or disadvantage in decision support systems, skyline queries are suitable for searching for solutions that are not dominated by each other in different dimensions. Therefore, we address a problem called MRRP with Time-Urgency (MRRP-TU). To effectively solve MRRP-TU, we propose an **MRRP-TU with multi-objective query anytime algorithm**. It combines priority queues, dominate pruning strategy, and potential cost estimation. Users can choose their favorite route in the solution set through the length of the route and delay time. In the experiment, we used the Tainan city's POI data. In the optimal solution algorithm, our method can significantly increase our queries' speed and show the ability to return solutions quickly. Experiments on approximate solutions also show that our method can return high-quality solutions in a short time.

Keyword: Multi-Request Route Planning, User Urgency, Multi-Objective Query, Location-based System

Content

中文摘要	I
Abstract	II
Content	III
List of Tables	V
List of Figures	VI
Chapter 1 Introduction	1
1.1 Background.....	1
1.2 Motivation	2
1.3 Problem Scenario.....	2
1.4 Research Purpose.....	3
1.5 Contribution.....	3
1.6 Organization	4
Chapter 2 Related Work	5
2.1 Single Dimensional Route Planning Problem	5
2.2 Multiple Dimensional Route Planning Problem.....	7
2.2.1 Single-Output Route Planning.....	7
2.2.2 Multiple-Output Skyline Query.....	9
Chapter 3 Problem Statement	12
Chapter 4 Methodology.....	16
4.1 Planning Flow.....	17
4.1.1 Skyline SkyS	17
4.1.2 Brute Force Search	19
4.1.3 Priority Queue	20

4.1.4	Dominate pruning strategy	23
4.1.5	A * estimate.....	27
4.1.6	Approximate algorithm	29
4.2	Algorithm	30
Chapter 5	Experimental Evaluation	34
5.1	Data Collection and Setting.....	34
5.2	Comparison of optimal algorithm.....	35
5.2.1	Impact of methodology.....	35
5.2.2	Impact of Number of Request NR	37
5.2.3	Impact of distance of Source and destination.....	38
5.2.4	Impact of Query Time	38
5.3	Comparison of greedy algorithm.....	39
5.4	Experimental Summary	42
Chapter 6	Conclusion and Future Work	44
References		46

List of Tables

Table 3-3-1 Summary of notations	12
Table 4-1 $SkyS$	17
Table 4-2 $Route' R'$	20
Table 4-3 $SkyS$	20
Table 4-4 $Queue$	20
Table 4-5 priority queue	21
Table 4-6 Priority Queue	22
Table 4-7 $SkyR', p$	24
Table 4-8 $SkyR', p$	26
Table 4-9 $SkyR', p$	27
Table 4-10 priority queue of estimate	28
Table 4-11 $SkyR'$	29
Table 5-1 parameters	35

List of Figures

Figure 1-1-1 A scenario of MRRP-TU	3
Figure 3-1 Example of MRRP-TU	13
Figure 4-1 Flowchart of one-way search algorithm	16
Figure 4-2 Skyline	17
Figure 4-3 Flowchart of BFS.....	19
Figure 4-4 Flowchart of Priority Queue	21
Figure 4-6 Example of dominate pruning strategy.....	24
Figure 4-5 Flowchart of dominate pruning strategy.....	24
Figure 4-7 Flowchart of A* estimate.....	28
Figure 4-8 Flowchart of approximate algorithm	29
Figure 5-1Number of POI Services	34
Figure 5-2 Impact of methodology.....	36
Figure 5-3 Convergence of methodology	37
Figure 5-4 Number of Expand and Dominate times	37
Figure 5-5 Impact of Number of Request.....	38
Figure 5-6 Impact of Distance of Source and Destination	38
Figure 5-7 Impact of Query Time.....	39
Figure 5-8 Numbers of POIs in different Time	39
Figure 5-9 Expand number of different Time.....	39
Figure 5-10 Quality of impact of k and methodology.....	40
Figure 5-11 Execution time of impact of k and methodology.....	41
Figure 5-12 convergence of methodology.....	41

Chapter 1 Introduction

1.1 Background

With the popularity of mobile devices and the development of wireless communication technologies and the Global Navigation Satellite System (GNSS), The application of Location-Based Service (LBS) is increasing rapidly. Among them, Point of Interest (POI) is the application of LBS. It marks locations on a map that people might be interested in, such as restaurants, gas stations, hospitals, etc. A POI contains at least latitude and longitude information, usually with a name, description, and attributes such as service, business hours, etc. Currently, public electronic map services such as Google Maps¹, Bing Maps², and Open Street Map³ (OSM) all provide POI query and route planning services. The route planning is where the user gives a starting point and a destination and several places to pass through on the way. The service will return one or more routes that are better in distance or time. In 2017 The Multi-Request Route Planning (MRRP) [11] was proposed. In the real world, a POI may have complex functions to satisfy multiple user needs. Route planning is a problem based on the needs of the user. For this query, the user provides the requests that want to be solved, e.g., a meal, a doctor's appointment, etc. The route planning service will select the POI that can solve the user's request on the map, and then form a route back to the user. In 2019 *SkyRoute* [37] was proposed, and is considered a kind of multidimensional MRRP problem. Depending on the number of different requests, the path distances return different routes. There are several other variants of the route planning problem in the world, but in all cases, the user is concerned with the accuracy of the query and the immediacy of the Timeliness. The route has the lowest possible travel costs of all the attributes, and the

¹ <https://www.google.com/maps>

² <https://www.bing.com/maps>

³ <https://www.openstreetmap.org/>

response time of the application cannot be too long. In terms of algorithm design, a lot of POIs and complete road network information are required to provide high-quality services. Because of the need to serve a large number of queries simultaneously, the computational cost cannot be too high, and these are the main challenges of route planning studies.

1.2 Motivation

In past studies of MRRP, because MRRP is an NP-hard problem, the algorithm design is based on approximate solutions found by the Greedy Algorithm. In MRRP, Each POI can provide more than one service to meet user needs, and This means that the problem may not find an optimal solution in the polynomial time, i.e., the shortest path. However, In the MRRP problem, time attribute is not considered, such as the business time of the POI and the user's urgency for different requests. In this study, we hope to find a solution that retains the ability to respond to inquiries and consider multiple attributes quickly, but it can still find the optimal solution. A lot of MRRP-related studies proposed in the past, but no one has considered the MRRP problem with time attribute in our knowledge. In summary, we address a new type of query called Multi-Request Route Planning with User Urgency (MRRP-TU).

1.3 Problem Scenario

This section describes the problem scenario that we hope to solve. Figure 1-1-1 is a route planning scenario in which a student has been to school all day and is preparing to go home at 17:00. Before he goes back, he hopes to have food in 20 minutes, a movie in an hour, and a drink in 10 minutes. In MRRP problems, because the time factor has no consideration and only the road distance has a concern, an orange path is chosen. However, in MRRP-TU, because of the different advantages and disadvantages of road distance and user urgency, a short orange path and a red path, which can satisfy the user's requests within the user urgency, is available for the user to choose.

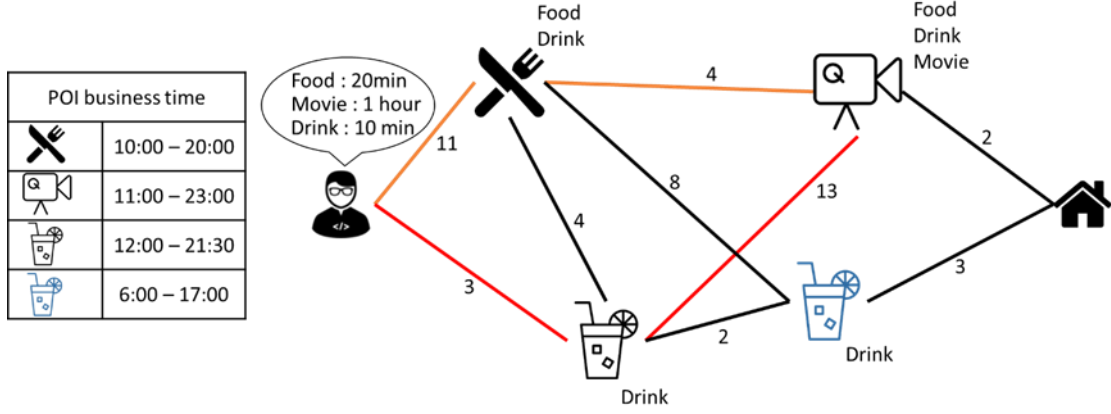


Figure 1-1-1 A scenario of MRRP-TU

1.4 Research Purpose

To get closer to the real world, we address a new type of query called Multi-request route planning with User Urgency, in this study, we consider the urgency of user requests and POI's business time. The urgency is whether user requests are satisfied within the time limit. The goal is to find a set of path solutions that consider the different dimensions of time and road distance. To solve this problem, we propose a skyline search algorithm to solve the MRRP-TU problem. The skyline search is an anytime algorithm, which returns feasible solutions quickly. We propose three different improvement methods to speed up the search for the best solution: priority queue, dominate pruning, and potential cost estimates. Since MRRP is a specific condition of MRRP-TU, MRRP is NP-hard, and MRRP-TU is also NP-hard. It is impractical to find and quickly return the optimal solution in the NP-hard problem, so we propose two greedy algorithms to speed up the search for feasible solutions.

1.5 Contribution

- (1) We address a new type of query called multiple request route planning with user urgency (MRRP-TU).
- (2) we propose a skyline search algorithm to solve the MRRP-TU.
- (3) Skyline search is an anytime algorithm. We use priority queues, dominate pruning,

and potential cost estimation to reduce the computation time and return feasible solutions consecutively until the optimal solution has found.

- (4) Based on the best solution algorithm, we propose an approximate solution algorithm that can return at an average time of 0.5 seconds in experimental terms, near 100% ratio approximate solution for quick answers to MRRP-TU questions.

1.6 Organization

The paper organized as follows. Chapter 2 reviews some studies related to route planning. Chapter 3 formally defines the problem to be addressed in this thesis. Chapter 4 introduces our proposed skyline search algorithm. Chapter 5 shows the experiment containing the optimal and approximate solutions of our algorithm and a comparison with a different method. Chapter 6 is the conclusion and future work.

Chapter 2 Related Work

In this section, we review some relevant research on route planning. The attribute characteristics of route considerations can divide into two broad categories: Single Dimensional Route Planning Problem, and Multiple Dimensional Route Planning Problem. The former only considers the single feature route planning problem. The latter considers the problem of path planning with multiple features and limitations. We will describe the current state of the research and describe the difficulties and limitations of these methods.

2.1 Single Dimensional Route Planning Problem

This type of study focuses on situations where the target function for route planning has only one feature. The post office problem is the most classical of these problems, where the residents find the post office closest to their home, and distance is the single dimension feature. In the route planning problem, the early discussion did not include providing the user with a route, but rather how to find points of interest in a route that already existed. In 2003, Jensen et al. proposed an approach to studying NN problems in graph theory and road networks[1]. Destination-oriented route planning began to be widely discussed between 2001 and 2010, to plan the optimal path for a given target function by selecting appropriate sections of the route for the destination. The Dijkstra algorithm [2] and the A* algorithm [3] are two classical shortest path optimal solution search algorithms. Dijkstra's algorithm uses the Priority Queue combined with Brand-First-Search to find the best solution in a network after a given starting point. Still, the execution time of the algorithm decreases as the network gets larger. Due to this drawback, the A* algorithm proposes to use an Estimate Function to evaluate the Path potential cost, which then speeds up the query for the shortest path planning. Multi-destination oriented path planning is a complex problem of combining path planning and alignment optimization. The classic problem is the Traveling Salesman Problem (TSP)

[4], the shortest path found in the network that allows the salesman to start from the starting point, visit all the destinations once, and then return to the starting point. TSP had proved to be NP-hard, and the search time will grow exponentially with the number of target points. Several heuristic approximations were proposing to reduce the execution time: the first one based on construction, which bases on the greedy algorithm—the method used to construct multi-destination paths incrementally. The second type is an improvement-based method, which based on the paths found so far. We are continually making improvements and adjustments. Albayraka et al. used a genetic algorithm and proposed the GSTM mutation operation [5] for TSP solution to improve the search efficiency. Dai et al. propose to use past user trajectory data for data exploration and recommend paths that meet users' preferences based on different user habits[6]. The problem of treating POIs on a road network as categories for doing route planning can be traced back to a 2005 paper. Li and others proposed the Trip Planning Query (TPQ) problem[7]. A category, for a group of categories, with the goal of selecting the most suitable POI for all categories between starting points and destination to Visiting. TPQ is a specific case of TSP, so all such problems are NP-Hard problems. Based on TPQ, Sharifzadeh et al. proposed an optimal sequence path query (OSR) technique[8], which progressively filter out the POIs that are not likely to be the optimal route to select the unfiltered POIs that are the shortest path from the destination to the starting point. In many real-world applications, travel planning queries are subject to several restrictions, such as the need to visit some POIs in order, and existing algorithms can only answer ordered or unrestricted. Chen et al. propose MRPSR[9], which allows only a partial collection of POIs to be visited in order during a trip. Based on the OSR, Liu et al. consider that users often have their preferences when choosing a route in the real world (KOSR). The OSR only provides a path to the user, but ignoring the user's preferences, so under the KOSR algorithm can find multiple paths composed of different POI's, and ensure that the paths can

satisfy the user's needs[10]. However, existing research treats POI as a type and then designs algorithms. Multi-Request Route Planning (MRRP) [11] proposed the concept of multiple requests, considering that a POI can satisfy multiple user requests simultaneously, and multiple demand path planning is proposed. The two modules are designed for this feature—first, the planning module, which uses a pruning and cache strategy to planning initial paths. Second, refining modules to further improve the quality of paths by reorganizing and replacing them. The paper concludes with a wide range of experiments confirm that the proposed algorithm is an improvement over the existing algorithm that does not consider the multiple requests provided by POI.

2.2 Multiple Dimensional Route Planning Problem

With the increase in the number of studies related to path planning issues, the problem has become more and more complicated in recent years. In order to bring the route planning problems closer to the real-world situation, more specific route planning problems are proposed. for example, Time-Dependent Road Network and Time Constraint. Our team calls this type of research a multi-dimensional route planning problem. It can divide into two research directions based on the format of the output: Single-Output Route Planning and Multiple-Output Skyline Query. The main difference between the two is that the latter does not integrate the feature of different values.

2.2.1 Single-Output Route Planning

This type of research focuses on the characteristics of the road network or the users' limitations and finds an appropriate path for providing users. Research to find points of interest in the process of moving(Continuous NN, CNN) is becoming increasingly important[12]. However, due to the lack of the concept of time parameterization, in a changing Spatial-temporal database. When the objects are moving, it is also necessary to consider the change of time and the change of other objects, so it is necessary to consider

the next query point from the current query point. Such problems are discussed and improved in [13]. Based on the data mining, Lu's research team proposed the PATE algorithm [14] to predict the travel time, and the subsequent consecutive path planning issues are of great interest. Lee et al. [15] used GPS to obtain real-time traffic information and provide user navigation services with real-time route planning. Lu et al. consider that route planning should consider the impact of traffic events on road congestion, and proposed the TEPA algorithm [16] to mining the correlation of traffic events and use it to predict the following traffic events, continually updating the fastest path planning results. Xu et al. [17] also believe that the real road network conditions are high-frequency changes. The authors analyze the road network traffic conditions, mining the knowledge of time and space to guide users to plan more effective paths and propose various techniques to avoid unnecessary calculations to enhance planning efficiency. However, there may be more than one destination in real navigation applications, and the optimal route must be arranged according to the relationship between the nodes. Hefez et al. proposed the TARS system [18], which allows users to enter time constraints and node visit sequences. Demiryurek et al. proposed the two-way A* search on the dynamic network [19].

Ma et al. proposed the dynamic network BESTMTNN [20] query. They consider that in the real world, users usually need to give the departure time and route guidance for each location at a given time period, which makes it necessary to consider the network's spatial and temporal characteristics when designing the algorithm. Cao et al. proposed an algorithm based on the keyword query to find a way to satisfy the user's time constraint [21]. Costa et al. extend the static road of OSR queries to a time-dependent road network. They proposed OTDSR queries [22], and extend the PNE method of OSR to become a time-dependent TD-PNE algorithm to compare with the A* search-based TD-OSR algorithm proposed in this paper. Huang et al. consider that most existing multi-destination route planning focuses on

minimizing travel costs, such as distance or time. But in reality, users may have multiple requirements simultaneously, such as reaching a destination within time constraints and minimizing travel costs, so they propose the MDRP-DCC [23] with time duration and cost constraints. The authors developed two methods to answer the MDRP-DCC question, the first of which was based on user-specified time period pruning, and thus The basic method of reducing the search space BMDC. However, When applied to larger networks. They were incurring a higher computational overhead. This method uses a heuristic function to estimate whether the sub-route paths can complete the tour within time constraints. Finally, the method presented in this paper shows excellent results based on extensive experimental evaluation of real urban road networks. Luan et al. proposed by The problem of multi-destination path planning based on dynamic road networks and time constraints[24]. Li et al. considered that MRRP would visit too many vertices, edges, and POIs if it is conducted on a real road network in a city in multi-demand path planning. However, these POIs are unimportant to the user, so they believe that the scope of the user's query does not need to consider all road networks' range, but to determine the scale of the search based on the user's starting point and demand. Based on this problem, Li proposed the GHMRPF[25]. They build an index structure of the POI for the network structure and consider the dynamic network to find the matching POI in the initially given range. If there is no POI in this range that meets the user's needs, then further expand the search area. Since the area to be searched is a real road subarea, it speeds up the search.

2.2.2 Multiple-Output Skyline Query

With Single-Output Route Planning, scholars are beginning to find that it is not appropriate to weigh all features in most cases. Weight Combination, where previous studies encounter different unit characteristics, will be used to evaluate each eigenvalue given a ratio and then used in some way: summing and multiplication. However, such calculations are

often subjective to the research team that the route provided to ignore user preferences. As a result of these problems, the Skyline Query was introduced in 2001 in order to solve multi-featured query problems. In 2001, Borzsonyi et al. proposed the basic concepts and operations of skyline queries in database systems, intending to answer the query problem in multiple dimensions. For example, how to choose the nearest hotel to the beach. A Skyline query will find a group of hotels because the price of the hotel and its distance from the beach will be taken into account when choosing a hotel. These hotels do not dominate each other. Dominate is defined as when an object dominates another object if it is as good in all dimensions, and better in at least one dimension[26]. In 2004, Xuegang et al. proposed how to use nearest neighbor queries in a database system for location-based services. The POI around a route combined with a skyline[27]. However, the search is still limited to database systems, so in 2006 Sharifzadeh et al. proposed This study examines how to perform skyline queries in Spatial Space, using R Tree index structure, partitioning accelerates skyline query problems in physical space[28]. Hung et al. were the first to consider how to perform skyline lookups in the presence of moving objects[29]. Although the studies mentioned above solved the problem of skyline lookup routes and the use of skyline queries in physical space, respectively, based on the road network structure, the skyline query question is still somewhat lacking. Therefore, in 2007, Deng et al. first raised the problem of skyline query in the road network structure. Skyline queries considered Euclidean distances. Zheng et al. proposed to consider the specific range when the object is moving[30]. This kind of study could not find the optimal solution because of the specific scope considered[31].

The skyline query is often used in path planning when the POI contains multiple features. Kriegel et al. considered that in addition to the POI's multiple features, the connection between road networks also contains multiple weights. Such as path length and path time, although the weight of the road network will have some dependence in most cases,

this is not absolute, e.g., a long detour in traffic congestion can reach the destination faster [32]. Huang et al. proposed a skyline query when moving to a destination[33]. For example, the number of cars on a given day will affect air quality. Therefore they are based on the following characteristics A related skyline query is designed for such problems[34]. Michael et al. have proposed to consider different values of features in different situations. Since the more features there are in a skyline query, the less number of solutions will be found, so they argue that users can choose the feature they care about for a skyline query[35]. Qixu et al. consider that people interact with each other in real life, so we cannot consider only one user but all users when we conduct a skyline query[36]. Yawalkar et al. have proposed methods for considering skyline queries to speed up MRRP queries and proposed queries without solving all the requirements based on user preferences[37].

Chapter 3 Problem Statement

In this section, we first introduce the symbols used in the studies, then introduce the definition of MRRP-TU, and finally, we prove that this problem is NP-hard. Summary of notation is the Summary of notation.

Table 3-3-1 Summary of notation

Symbol	Meaning
s	Start point
e	Destination
s_t	Start time
N_R	The number of requests
N_S	The number of services in the data set
N_P	The number of POIs in the data set
R	The collection of requests, where $R = \{r_1, r_2, \dots, r_{N_R}\}$
D	The collection of deadline of request r
S	The collection of services, where $S = \{s_1, s_2, \dots, s_{N_S}\}$
P	The collection of all POIs, where $P = \{p_1, p_2, \dots, p_{N_P}\}$
$B(p)$	The POI business time
$Route$	A $Route = (p_{x_1}, p_{x_2}, \dots, p_{x_m})$, where p_{x_i} is a POI
$d(p_{x_i}, p_{y_j})$	The distance between p_{x_i} and p_{y_j}
$t(p_{x_i}, p_{y_j})$	The travel time between p_{x_i} and p_{y_j}
T_{x_i}	The time when user reach POI p_{x_i}
$Delay(r)$	The delay time of request r
$Dis(Route)$	The distance of the route

$Delay(Route)$	The delay time of the route
$PSR(r)$	The set of all POIs that provide request r
$RSP(p)$	The set of all services that provided by POI p

Definition 1. *Request, Deadline and Services*

Requests $R = \{r_1, r_2, \dots, r_{N_R}\}$ represents the user's requests where $i \neq j$ and $r_i \neq r_j$. Requests deadline $D = \{d_1, d_2, \dots, d_{N_R}\}$ where $d_i = D(r_i), \forall i \in N_R$ represents the time limit for users to complete different request. $S = \{s_1, s_2, \dots, s_{N_S}\}$ where $i \neq j$ and $s_i \neq s_j$. $R \subseteq S$ and $N_R \leq N_S$ represents all services in the dataset. Take Figure 3-1 as example. $R = \{r_1, r_2\}$, $D = \{d_1 = 30min, d_2 = 150min\}$. In this example, one unit of time for moving time is equal to one unit of distance. But in a real-life example, we can use the average speed per minute, or use the google API to calculate Get the correct travel time for the conversion.

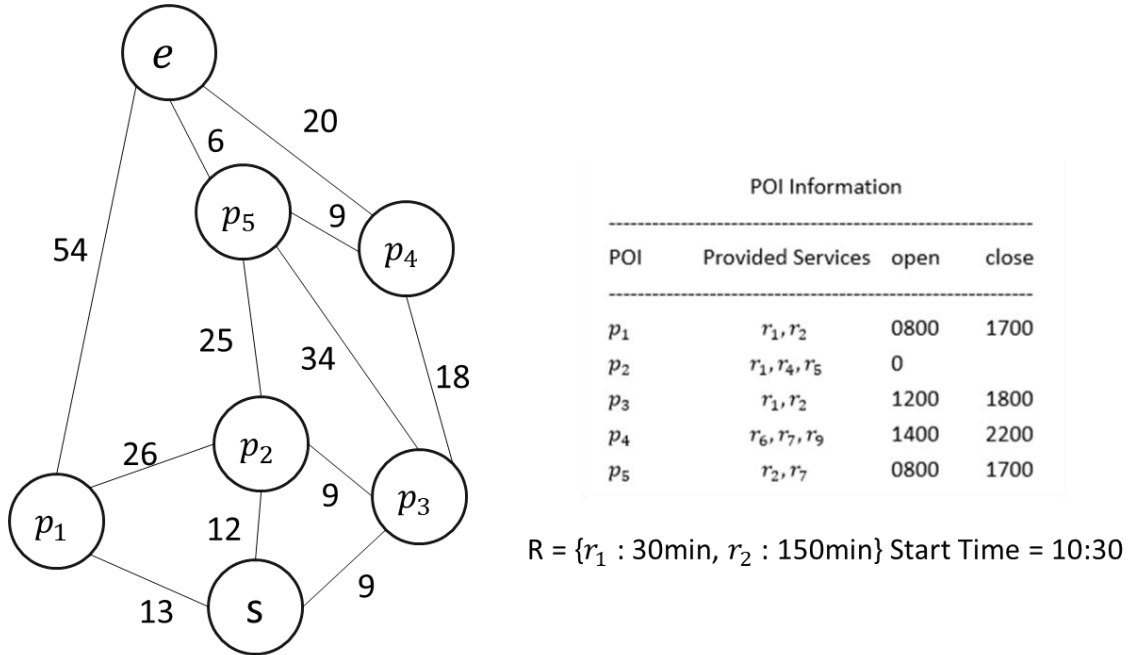


Figure 3-1 Example of MRRP-TU

Definition 2. *Route*

A *Route* $= (s, p_{x_1}, p_{x_2}, \dots, p_{x_m})$ is an ordered series consisting of one or more POIs,

where $1 \leq x_i \leq N_p$, $x_1 \neq x_2 \neq \dots \neq x_m$.

Definition 3. *Current time and Delay time of request*

Given a $Route = (s, p_{x_1}, p_{x_2}, \dots, p_{x_m})$, Current time T_{x_i} is the time when the user reaches POI p_{x_i} . The formula is shown in equation (1).

$$T_{x_i} = t(s, p_{x_1}) + \sum_{k=2}^i t(p_{x_{k-1}}, p_{x_k}), \text{ where } 1 \leq x_i \leq N_p, 1 \leq i \leq m \quad (1)$$

Delay time $Delay(r)$ is the delay time for request r . When $r \in R \cap RSP(p_{x_i})$, The $Delay(r)$ formula is shown below.

if $B(p_{x_i}) > T_{x_i}$ **and** $D(r) < B(p_{x_i})$, $Delay(r) = 0$

else if $B(p_{x_i}) > T_{x_i}$ **and** $D(r) > B(p_{x_i})$, $Delay(r) = D(r) - B(p_{x_i})$

else if $B(p_{x_i}) \leq T_{x_i}$ **and** $T_{x_i} < D(r)$, $Delay(r) = 0$

else if $B(p_{x_i}) \leq T_{x_i}$ **and** $T_{x_i} > D(r)$, $Delay(r) = T_{x_i} - D(r)$

Definition 4. *Distance and Delay cost*

Given a $Route = (s, p_{x_1}, p_{x_2}, \dots, p_{x_m})$, $Dis(Route)$ and $Delay(Route)$ are defined as the length of the route and the cost of delay. R_{solved} is the request solved by point p_{x_i} . $Dis(Route)$ and $Delay(Route)$ formulas are shown in equation (2) (3).

$$Dis(Route) = d(s, p_{x_1}) + \sum_{k=2}^i d(p_{x_{k-1}}, p_{x_k}), \quad (2)$$

where $1 \leq x_i \leq N_p, 1 \leq i \leq m$

$$Delay(Route) = \sum_{i=1}^m \sum_{\forall r \in R_{solved}} Delay(r) \quad (3)$$

$$\text{where } R_{solved} = \left(R - \bigcup_{1 \leq j \leq i-1} RSP(p_{x_j}) \right) \cap RSP(p_{x_i})$$

Definition 5. *PSR and RSP*

Given a request $r \in R$ and a POI $p \in P$, $PSR(r)$ is defined to be all services providing r POI set, and $PSR(r) \subseteq P$. $RSP(p)$ is defined as the set of all services provided to p . and $RSP(p) \subseteq S$.

Definition 6. *Valid Route*

Give a set of user-specified requests $R = \{r_1, r_2, \dots, r_{N_R}\}$, and a $Route = (s, p_{x_1}, p_{x_2}, \dots, p_{x_m}, e)$, $Route$ is said to be a valid path if and only if $R = \bigcup_{i=1}^m RSP(p_{x_i})$. In other words, a valid route means that it satisfies all the requests specified by the user.

Definition 7. *Multi-Request Route Planning with User Urgency (MRRP-TU)*

This section recapitulates the definition of MRRP-TU. Given all POI datasets $P = \{p_1, p_2, \dots, p_{N_P}\}$, a start point s , query time s_t , and an destination e . and a set of user-specified requests $R = \{r_1, r_2, \dots, r_{N_R}\}$, requests deadline $D = \{d_1, d_2, \dots, d_{N_R}\}$, The goal of MRRP-TU is to find a valid route collections $SkyS$ from the start point s to the destination e .

Theorem 1. *The MRRP-TU query is NP-hard*

Proof: In MRRP-TU problems, one POI can solve more than one request, and the user has a request deadline, the POI has business hours. Assume that the user request deadline is infinite; then assume that the POI is open for 24 hours, then the MRRP-TU is reduced to MRRP. Since MRRP is NP-hard, MRRP-TU is also NP-hard.

Research Objective: Based on the MRRP-TU problem, we divide the research goal into the optimal solution and approximate solution. The optimal solution is to find the skylines, and the execution time is as little as possible. An approximate solution aims to significantly reduce execution time at the expense of some route quality.

Chapter 4 Methodology

In this section, we propose two algorithms to answer the MRRP-TU query based on the best solution and the approximate solution. Figure 4-1 is a flowchart of the algorithm. In the optimal solution section, Skyline Search is an anytime algorithm. It searches for routes in the best first method. Each search by Breadth-first Search adds at least one unsolved request that can be solved. And then add the path to the priority queue to quickly return a set of feasible solutions. Skyline search is also a branch and bound method. It restarts the search multiple times to update the skyline on the POI, so each search reduces the number of paths to be searched for the next one, and when there are no paths in the priority queue, the Skyline is found, and the algorithm is stopped. In order to find feasible solutions in limited time, we reduced the number of selected POIs and relaxed the definition of dominate pruning strategy so that the skyline found is not the optimal solution, but is substantially faster in execution speed. The following section is divided into Skyline, BFS, Priority queue, Dominate pruning strategy, A* estimate, approximate, and pseudo-code.

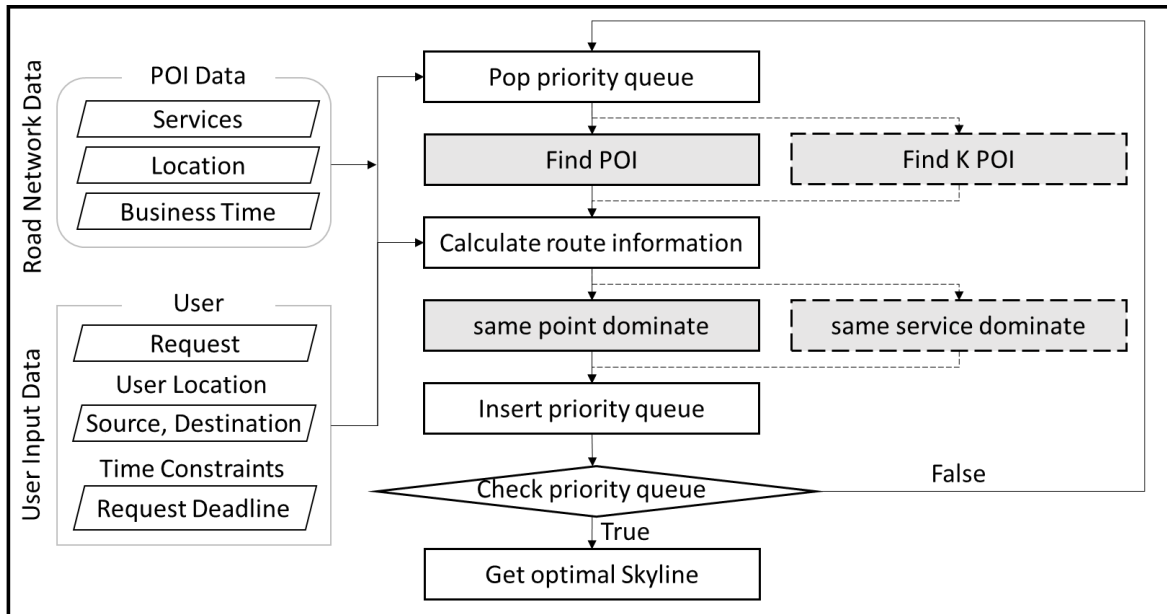


Figure 4-1 Flowchart of one-way search algorithm

4.1 Planning Flow

To solve MRRP-TU problems. This section will introduce step by step the algorithm design and improvement process. It starts with an introduction to finding the optimal solution using the brute force method and introduces the priority queue, skyline dominate and potential cost estimation, and finally propose the best solution algorithm based on the multi-objective query.

4.1.1 Skyline SkyS

There are two different attributes of the route found by MRRP-TU, the delay time, and the road distance. Unlike in the past, when we considered one dimension for route planning, therefore we used the concept of skyline dominate to find a multi-objective path solution for MRRP-TU. $dominate(Route1, Route2)$ represents the dominance operation of route over each other. Given that both paths, $Route1$ and $Route2$, solve all the requests, $Route1$ dominates $Route2$ if $Route1$ is as good in all dimensions, and better in at least one dimension. Take Figure 3-1 as an example. The set of paths we finally find is $SkyS$, as shown in Table 4-1 $SkyS$. We draw the route on the table as shown in Figure 4-2

Route	Delay , Distance
s, p_1, e	0 、 68
s, p_3, e	60 、 42
s, p_2, p_1, e	0 、 92
s, p_2, p_3, e	0 、 54
s, p_5, p_1, e	56 、 154
s, p_5, p_3, e	27 、 96

Table 4-1 $SkyS$

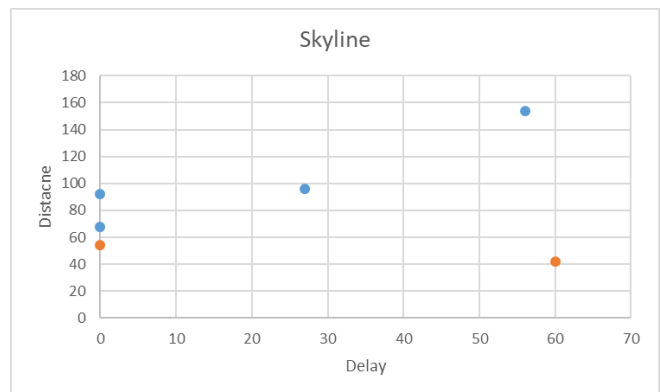


Figure 4-2 Skyline

As you can see, the orange point can be the skyline we find. Function1 is the pseudo-code of *dominate* operation. We dominate all routes in the *SkyS*.

dominate($Route_1, Route_2$)

$$= \begin{cases} \text{TRUE}, & \text{Dis}(Route_1) \geq \text{Dis}(Route_2) \text{ and } \text{Delay}(Route_1) \geq \text{Delay}(Route_2) \\ & \text{and } (\text{Dis}(Route_1) > \text{Dis}(Route_2) \text{ or } \text{Delay}(Route_1) > \text{Delay}(Route_2)) \\ \text{FALSE}, & \text{otherwise} \end{cases}$$

Function 1 *dominate*($Route_{Set}$)

Input: $Route_{Set}$

```

1:  For i = 0 to len( $Route_{Set}$ )
2:    For j = 0 to len( $Route_{Set}$ )
3:      If dominate( $Route_{Set}[i], Route_{Set}[j]$ )
4:        Remove  $Route_{Set}[j]$ 

```

4.1.2 Brute Force Search

To solve the MRRP-TU problem. The most basic approach is the Brute Force Search algorithms (BFS); the core concept of BFS is searching all possibilities through all the alignments in the path. BFS can find the optimal solution. Figure 4-3 is a flowchart of BFS. Here are the detailed steps of the BFS.

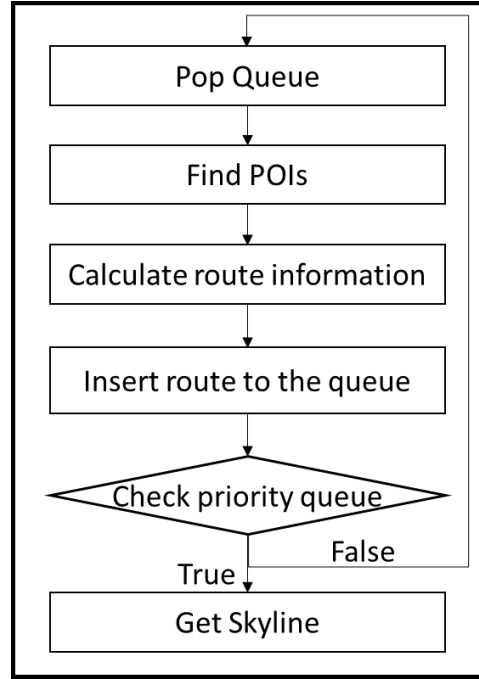


Figure 4-3 Flowchart of BFS

- Step 1.** BFS pop start points s or $Route$ from the queue, R' represent solved request by $Route$ where $R' = R \cap \cup_{p \in Route'} RSP(p)$. Find all $POIs$ that solve at least one request in $R_{unslove}$. $POIs = \{p_1, p_2, \dots, p_k\}$ $\circ R_{unslove} = R - R'$, $p_i \in \cup_{r \in R_{unslove}} PSR(r), \forall 1 \leq i \leq k$.
- Step 2.** BFS generates new $Route' = \{p_{x_1}, p_{x_2}, \dots, p_{x_m}, p\}$ for all p in $POIs$, and calculates the solution requirement R' for each path.
- Step 3.** BFS determines the R' of the $Route'$, When $|R'| = N_R$ then connect $Route'$ back to the destination e and calculate the distance and delay time of

$Route'$, $Dis(Route') \cdot Delay(Route')$. Finally, add the $Route'$ to the skyline $SkyS$. When $|R'| \neq N_R$, add the $Route'$ to queue

Step 4. BFS repeats steps 1 to 3 until the queue is empty. When the queue is empty, BFS $dominate(SkyS)$ and finally outputs the optimal solution for this query.

POI	R'
p_1	$\{r_1, r_2\}$
p_2	$\{r_1\}$
p_3	$\{r_1, r_2\}$
p_5	$\{r_2\}$

Table 4-2 $Route'$ R'

$SkyS$
$\{0, 68, \{s, p_1, e\}\}$

Table 4-3 $SkyS$

$Queue$
$\{\{s, p_3\}\}$

Table 4-4 $Queue$

We use Figure 3-1 for example. Step 1. The user finds all POIs p_1 , p_2 , p_3 , and p_5 that satisfy at least one of the user's requests from the starting point s . Step 2. Calculate the solved requests R' for the found $Route'$ in order, as shown in Table 4-2. Step 3. Check the $|R'|$ for all $Route'$. Take p_1 as an example, $|R'| = N_R$, therefore we generate $Route' = \langle s, p_1, e \rangle$ then add $Route'$ to $SkyS$ as shown in Table 4-3. Take p_2 as an example, $|R'| \neq N_R$, therefore we add $Route'$ to $Queue$ as shown in Table 4-4. Step 4. Take p_3 as an example, the queue will pop $Route = \langle s, p_3 \rangle$. Repeat steps 1 to 3, when $Queue = \{\}$ means the end of search, then $dominate(SkyS)$ to get the skyline.

4.1.3 Priority Queue

In the previous section, we introduced BFS. In the BFS, we can see that there is only the basic concept of First In First Out (FIFO) in the queue iteration process. The queue has no sorting ability and may not be able to find the solution fast enough when expanding routes. Still, due to the anytime algorithm, we want to output the solution at every moment. Therefore, we think that the priority queue can help us to find the most suitable path faster.

The priority queue sorts the different paths and gives them priority to output the solutions that we think are better according to our requests. In addition to the priority queue, we also think that the number of solution requests also affects our algorithm search speed. Therefore, we have designed a data structure combining the priority queue and the number of solved requests.

Table 4-5 is an abstraction of the priority queue. *sloved requests* represents the number of solved requests $|R'|$, where $k \leq N_R$. *priority* representing the priority queue under different number of solved requests. We will first take the delay time as the first value and the distance as the second when expanding route. When the delay time is shorter, the priority queue will expand the route first.

<i>sloved requests</i>	<i>priority</i>
1	$\{\{Delay(Route), Dis(Route), Route\}\}$
:	$\{\{Delay(Route), Dis(Route), Route\}\}$
k	$\{\{Delay(Route), Dis(Route), Route\}\}$

Table 4-5 priority queue

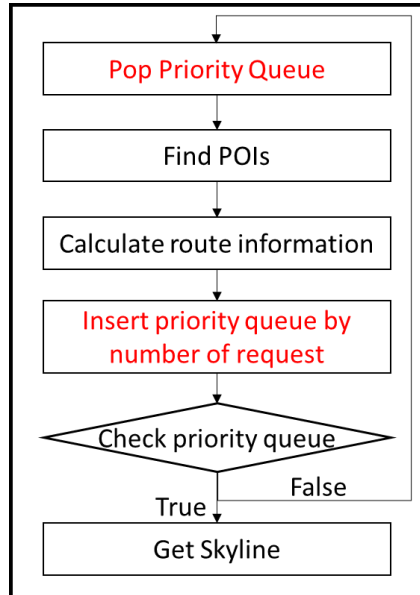


Figure 4-4 Flowchart of Priority Queue

We can use this data structure for expanding the path to solving a high number of requests,

short delay, and a short length. In this way, we can find a route that meets the request faster. Figure 4-4 shows the Priority Queue flowchart. Here are the detailed steps for Priority Queue.

- Step 1.** Priority Queue pop start points s or $Route, R'$ represent solved request by $Route$ where $R' = R \cap \cup_{p \in Route'} RSP(p)$. Check if $|R'| = N_R$, then generate $Route'$ that connect to destination e , and add to $SkyS$. If $|R'| \neq N_R$ Find all $POIs$ that solve at least one request in $R_{unslove}$. $POIs = \{p_1, p_2, \dots, p_k\} \circ R_{unslove} = R - R'$, $p_i \in \cup_{r \in R_{unslove}} PSR(r), \forall 1 \leq i \leq k$.
- Step 2.** generates new $Route' = \{p_{x_1}, p_{x_2}, \dots, p_{x_m}, p\}$ for all p in $POIs$, and calculates the solution requirement R' for each path.
- Step 3.** Calculate the R' of the $Route'$. $Dis(Route'), Delay(Route')$ from $Route'$, and add $Route'$ to the priority queue according to R' , $Dis(Route')$, and $Delay(Route')$.
- Step 4.** repeats steps 1 to 3 until the priority queue is empty. When the priority queue is empty, algorithm execute $dominate(Skys)$ and finally outputs the optimal solution for this query.

Take Figure 1 as an example, Continuing from section 4.1.2, Step 3. After calculating all the requests R' we can get $p_1 = \{r_1, r_2\}$, $p_2 = \{r_1\}$, $p_3 = \{r_1, r_2\}$, $p_5 = \{r_2\}$, according to R' we can get new Priority Queue:

$$Priority\ Queue = \left\{ 0: \{\}, 1: \left\{ \{0, 12, < s, p_2 >\}, \{0, 36, < s, p_5 >\} \right\}, 2: \left\{ \{0, 13, < s, p_1 >\}, \{60, 9, < s, p_3 >\} \right\} \right\}$$

The Priority Queue is shown in Table 5.

Priority Queue	
0	$\{\}$
1	$\{0, 12, \{s, p_2\}\}, \{0, 36, \{s, p_5\}\}$
2	$\{0, 13, \{s, p_1\}\}, \{60, 9, \{s, p_3\}\}$

Table 4-6 Priority Queue

Step 4. Repeat steps 1 to 3 until Priority Queue is empty. This means all paths are searched, and then $dominate(Skys)$ to get the skyline. From the above example, we can see that for the paths $\langle s, p_2 \rangle, \langle s, p_3 \rangle$, if the priority queue is not used, $\langle s, p_2 \rangle$ will expand first, making the speed of finding the solution slow down.

4.1.4 Dominate pruning strategy

4.1.2 We show how to solve MRRP-TU problems in BFS, 4.1.3 combine the priority queue speeds up the algorithm to find solutions. But even with the priority queue, the number of expansions in a single query is still the same as in BFS, and the only difference is that we can export some of the solutions earlier. Therefore, to speed up the solution finding, we designed a new pruning strategy called dominate pruning based on the MRRP-TU characteristics. In the MRRP-TU problem, our goal is to find a set of skylines. All the paths within this skyline can satisfy the user's needs and are not dominated by each other. From the above description, we can see that the skyline that satisfies the user's requests is a path that connects from the start point to the destination. Based on this feature, we propose the dominate pruning strategy. The following is definition of dominate pruning. Given an MRRP-TU query $(s, e, s_t, R = \{r_1, \dots, r_{N_R}\}, D = \{d_1, \dots, d_{N_R}\})$, If two paths :

$$Route_1 = (s, p_{x_1}, p_{x_2}, \dots, p_{x_m}, p_{x_c}), Route_2 = (s, p_{y_1}, p_{y_2}, \dots, p_{y_n}, p_{y_c})$$

where $1 \leq x_i, y_j \leq N_p, 1 \leq i \leq m, 1 \leq j \leq n, x_1 \neq x_2 \neq \dots \neq x_m, y_1 \neq y_2 \neq \dots \neq y_n$

If $Route_1 R' = Route_2 R'$, $p_{x_c} = p_{y_c}$. Then $Route_1, Route_2$ can do the skyline operation. If $dominate(Route_1, Route_2) = TRUE$, then $Route_2$ does not need to expand.

Here is the proof of the dominate pruning. Assume $Route_1 = (s, p_{x_1}, p_{x_2}, \dots, p_{x_m}, p_{x_c})$ and

$Route_2 = (s, p_{y_1}, p_{y_2}, \dots, p_{y_n}, p_{y_c})$. If the $Route'_1 = (s, p_{x_1}, p_{x_2}, \dots, p_{x_m}, p_{x_c}, p_{x_{c+1}}, \dots, e)$

exists, and $Route'_1 R' = R$, is the optimal route for $Route_1$ to solve all requests.

$Route'' = (p_{x_{c+1}}, \dots, e)$ is the path from $p_{x_{c+1}}$ to destination e . Then optimal route for

$Route_2$ to solve all requests $Route'_2 = (s, p_{y_1}, p_{y_2}, \dots, p_{y_n}, p_{y_c}, p_{y_{c+1}}, \dots, e)$.

The delay distance is as follow:

$$Dis(Route'_1) = Dis(Route_1) + Dis(Route''),$$

$$Dis(Route'_2) = Dis(Route_2) + Dis(Route''),$$

$$Delay(Route'_1) = Delay(Route_1) + Delay(Route''),$$

$$Delay(Route'_2) = Delay(Route_2) + Delay(Route'')$$

Because $dominate(Route_1, Route_2) = TRUE$, $dominate(Route'_1, Route'_2) = TRUE$.

Figure 4-6 is an example: There are two paths from s to p , $Route_1$, $Route_2$, they both solve the R' requests, the delay time and distance is 10,10 for $Route_1$ and 11,11 for $Route_2$. Assume that there exists an optimal route from p to destination e that solves the remaining unresolved requests because $dominate(Route_1, Route_2) = TRUE$, so based on the above proof $Route_2$ does not need to be expanded.

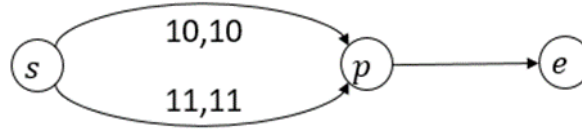


Figure 4-6 Example of dominate pruning strategy

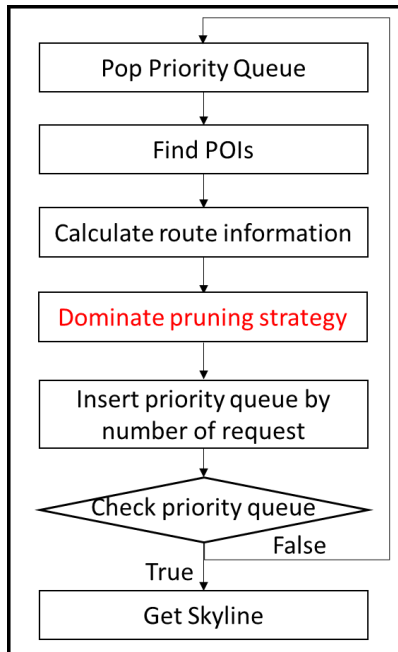


Figure 4-5 Flowchart of dominate pruning strategy

$Sky(R', p)$	$\{Route\}$
R'_i, p_i	$\{Route_{i_1}, Route_{i_2}, \dots, Route_{i_m}\}$
R'_j, p_j	$\{Route_{j_1}, Route_{j_2}, \dots, Route_{j_m}\}$

Table 4-7 $Sky(R', p)$

We define a pruning strategy with the above description: when two paths address the same requests and reach the same point. Two paths are available for dominate pruning strategy. In this way, we have added a skyline structure to each POI. $Sky(R', p) = \{Route_1, Route_2, \dots, Route_m\}$. This way we can remove the paths in each POI that do not need to be expanded. Table 4-7 is a data structure. Figure 4-4 is a flowchart for adding a dominate pruning strategy. The following steps are described in detail.

- Step 1.** Priority Queue pop start points s or $Route, R'$ represent solved request by $Route$ where $R' = R \cap \cup_{p \in Route} RSP(p)$. Check if $|R'| = N_R$, then generate $Route'$ that connect to destination e , and add to $SkyS$. If $|R'| \neq N_R$ Find all $POIs$ that solve at least one request in $R_{unslove}$. $POIs = \{p_1, p_2, \dots, p_k\} \circ R_{unslove} = R - R'$, $p_i \in \cup_{r \in R_{unslove}} PSR(r), \forall 1 \leq i \leq k$.
- Step 2.** generates new $Route' = \{p_{x_1}, p_{x_2}, \dots, p_{x_m}, p\}$ for all p in $POIs$, and calculates the solution requirement R' for each path.
- Step 3.** For all $Route'$, calculates the R' of the $Route'$, distance, and the delay time.
 $Dis(Route') \cdot Delay(Route')$
- Step 4.** Add $Route'$ to $Sky(R', p)$ based on R' and POI p of $Route'$, dominate all $Sky(R', p)$
- Step 5.** Adding non-eliminated $Route'$ according to R' , $Delay(Route')$ and $Dis(Route')$ to the priority queue.
- Step 6.** repeats steps 1 to 5 until the priority queue is empty. When the priority queue is empty, algorithm execute $dominate(Skys)$ and finally outputs the optimal solution for this query.

Take Figure 1 as an example, Continuing from section 4.1.2, Step 3. After calculating all the requests R' we can get $p_1 = \{r_1, r_2\}$, $p_2 = \{r_1\}$, $p_3 = \{r_1, r_2\}$, $p_5 = \{r_2\}$, Step 4 .according to R' and p we can add $Route'$ to $Sky(R', p)$ As shown in Table 4-8:

$$Sky(R', p) = \left\{ \begin{array}{l} r_1, r_2, p_1: \{\{0,13, < s, p_1 >\}\}, \\ r_1, p_2: \{\{0,12, < s, p_2 >\}\}, \\ r_1, r_2, p_3: \{\{60,9, < s, p_3 >\}\}, \\ r_2, p_5: \{\{0,36, < s, p_5 >\}\} \end{array} \right\}$$

$Sky(R', p)$	<i>Route</i>
r_1, r_2, p_1	$\{\{0,13, < s, p_1 >\}\}$
r_1, p_2	$\{\{0,12, < s, p_2 >\}\}$
r_1, r_2, p_3	$\{\{60,9, < s, p_3 >\}\}$
r_2, p_5	$\{\{0,36, < s, p_5 >\}\}$

Table 4-8 $Sky(R', p)$

Since every $Sky(R', p)$ doesn't have more than two route, no route is eliminated. Step 5.

Add all the routes to the priority queue. $Priority\ Queue = \{0: \{\}, 1: \{\{0,12, < s, p_2 >\}, \{0,36, < s, p_5 >\}\}, 2: \{\{0,13, < s, p_1 >\}, \{60,9, < s, p_3 >\}\}\}$. Back to step 1, we can get $\{0,13, < s, p_1 >\}, \{60,9, < s, p_3 >\}$ and connect it back to the destination e . At this time $Priority\ Queue = \{0: \{\}, 1: \{\{0,12, < s, p_2 >\}, \{0,36, < s, p_5 >\}\}\}$, We can get $Route = \{0,12, < s, p_2 >\}$ from the priority queue. From the last visited POI p_3 , the algorithm looks for at least one requested POIs, p_1 and p_4 , which can be resolved. Step 2. Generate a new $Route' = \{0,32, < s, p_2, p_1 >\} \cdot \{0,21, < s, p_2, p_3 >\}$. Step 3 and 4. add the new path to $Sky(R', p)$ As shown in Table 4-9:

$$Sky(R', p) = \left\{ \begin{array}{l} r_1, r_2, p_1: \left\{ \begin{array}{l} \{0,13, < s, p_1 >\}, \\ \{0,32, < s, p_2, p_1 >\} \end{array} \right\}, \\ r_1, p_2: \{\{0,12, < s, p_2 >\}\}, \\ r_1, r_2, p_3: \left\{ \begin{array}{l} \{60,9, < s, p_3 >\}, \\ \{0,21, < s, p_2, p_3 >\} \end{array} \right\}, \\ r_2, p_5: \{\{0,36, < s, p_5 >\}\} \end{array} \right\}$$

$Sky(R', p)$	$\{Route\}$
r_1, r_2, p_1	$\left\{ \begin{array}{l} \{0,13, < s, p_1 >\} \\ \{0,32, < s, p_2, p_1 >\} \end{array} \right\}$
r_1, p_2	$\{\{0,12, < s, p_2 >\}\}$
r_1, r_2, p_3	$\left\{ \begin{array}{l} \{60,9, < s, p_3 >\} \\ \{0,21, < s, p_2, p_3 >\} \end{array} \right\}$
r_2, p_5	$\{\{0,36, < s, p_5 >\}\}$

Table 4-9 $Sky(R', p)$

Then applied dominate pruning strategy to all $Sky(R', p)$. Take $r_1, r_2, p_1: \{\{0,13, < s, p_1 >\}, \{0,32, < s, p_2, p_1 >\}\}$ for example, the route $\{0,32, < s, p_2, p_1 >\}$ no need for further expansion. Because it will be dominated by $\{0,13, < s, p_1 >\}$. Take $r_1, r_2, p_4: \{\{60,9, < s, p_3 >\}, \{0,21, < s, p_2, p_3 >\}\}$ for example. The route $\{0,21, < s, p_2, p_3 >\}$ can continue to expand. Because it cannot be dominated by $\{60,9, < s, p_3 >\}$.

Step 5. Add the un-dominated paths to the *Priority Queue*:

$$Priority\ Queue = \left\{ 0: \{\}, 1: \{\{0,12, < s, p_2 >\}, \{0,36, < s, p_5 >\}\}, 2: \{\{0,21, < s, p_2, p_3 >\}\} \right\}.$$

Step 6. Repeat steps 1 to 5 until Priority Queue is empty. This means all paths are searched, and then $dominate(Skys)$ to get the skyline. Through the dominate pruning strategy, we can eliminate paths that do not require further expansion, thereby speeding up the process of finding solutions.

4.1.5 A* estimate

In 錯誤! 找不到參照來源。 our method already works well for MRRP-TU. Still, in the priority queue for selecting the paths When expanding, we only consider the length of the current path without considering the potential cost of the path to reach its destination. Therefore, in this section, we introduce the *Priority A* estimate* of the priority queue after considering the potential cost. In the route planning problem, A* has been a very sophisticated method, through A* we can estimate the potential cost of the path, and then

select the path with the lower potential cost. The path can be expanded faster towards the destination. We select paths worth expanding by adding A* to the priority queue. And because A* considers the potential cost of the overall path, we take away the limit on the number of solved requests. Given POI p , $estimate(p) = d(p, e)$. The Figure 4-7 and Table 4-10 below shows the process of joining A* estimate and the priority queue.

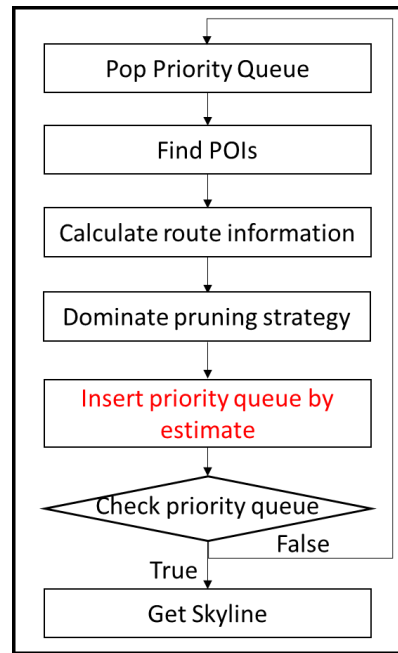


Figure 4-7 Flowchart of A* estimate

Priority
$\{(Delay(Route), Dis(Route) + estimate(p), Route)\}$
$\{(Delay(Route), Dis(Route) + estimate(p), Route)\}$
$\{(Delay(Route), Dis(Route) + estimate(p), Route)\}$

Table 4-10 priority queue of estimate

4.1.6 Approximate algorithm

In 4.1.2 to 4.1.5, We propose the algorithm of the MRRP-TU problem in an optimal solution. However, MRRP-TU is an NP-Hard problem, so finding the optimal solution, while good, is not practical. Therefore, we hope that the algorithm can return a suitable path solution in a few seconds. In summary, we move from the study of optimal solutions to the study of approximate solutions. It is expected that the query time can be significantly reduced in a condition close to the optimal solution quality. For the first approximation, we refer to the greedy algorithm of previous path planning, k nearest candidate (k NC). And considering the A*estimate method we use, we propose kA^* to speed up our query. A second approximation is to relax the constraints of the dominate pruning strategy. In the previous sections, we presented that the dominate pruning strategy can be used to solve two paths for the same service and the same last points. In the approximate solution, we remove the restriction that two paths need to have the same last point $Sky(R', p)$, allowing a skyline pruning strategy even for paths with different points $Sky(R')$, which eliminates more paths but also sacrifices the possibility of finding the optimal solution. Figure 4-8 is the flowchart

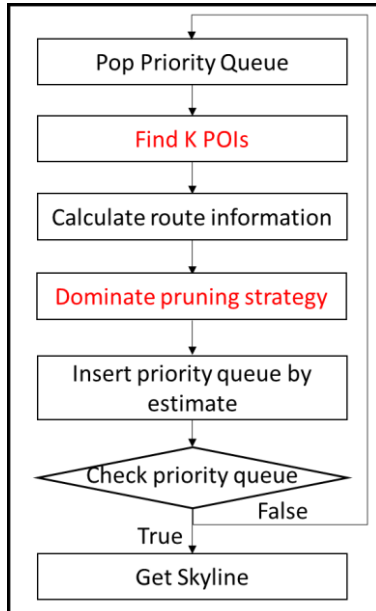


Figure 4-8 Flowchart of approximate algorithm

$Sky(R')$	$\{Route\}$
R'_i	$\{Route_{i_1}, Route_{i_2}, \dots, Route_{i_m}\}$
R'_j	$\{Route_{j_1}, Route_{j_2}, \dots, Route_{j_m}\}$

Table 4-11 $Sky(R')$

of the approximate solution and Table 4-11 is the transformation and introduction of the structure.

4.2 Algorithm

We have introduced all the improvements to the algorithm from the priority queue, skyline dominate, to A* estimate in the previous sections. This section will introduce a combination of all methods. The pseudo-code of the optimal solution is shown in Algorithm1. Lines 1 to 3 are initialization, and the starting point s is placed in the priority queue as a path. Line 4 of the code represents the stop condition of the algorithm. Line 5 is to pop the *Route* from the priority queue Q and prepare it for expansion. Lines 6 to 9 are to check if the *Route* has solved all the requests. Add the *Route* to the *Skys* if all requests have been satisfied. Lines 11, 12 is to find all that can be at least solve a request POI p . Lines 13 to 17 are to calculate the delay time and distance RI of the *Route'*. and added RI to $Sky(R', p)$. Lines 18 through 23 are a dominate pruning strategy for $Sky(R', p)$ and add the RI' that was not eliminated, then add to Priority queue added after A* estimate. Line 24 will output the optimal solution. Since our algorithm is an anytime algorithm, each time we find a solution that satisfies the requests, we can also do the following *dominate(Skys)* to get the algorithm's current solution.

Algorithm 1 Skyline Search

Input: s (starting point), s_t (Start time), e (destination), R (the collection of requests), D (The collection of requests deadline)

Output: *Skys*

- 1: $SkyS = \{\}, Priority\ Queue\ Q = \{\}, Sky(R', p) = \{\}$
 - 2: $Route = \langle s \rangle$
 - 3: $Priority\ Queue\ Q.insert(\{Delay(Route), Dis(Route), Route\})$
-

```

4:  While  $Q \neq \{\}$ 
5:     $Route \leftarrow Q.pop()$ 
6:    If  $Route.R' == |R|$ 
7:       $Route' = Route + e$ 
8:       $SkyS.insert(\{Delay(Route'), Dis(Route'), Route'\})$ 
9:      continue
10:    $POI_{List} = []$ 
11:   for  $POI\ p, p \in \cup_{r \in R} PSR(r), \forall 1 \leq i \leq k$  do
12:      $POI_{List}.insert(p)$ 
13:     for each  $p$  in  $POI_{List}$  do
14:        $Route' = Route + p$ 
15:        $R' = R \cap \cup_{p \in Route'} RSP(p)$ 
16:        $Route_{information}\ RI = \{Delay(Route'), Dis(Route'), Route', 0\}$ 
17:        $Sky(R', p).insert(RI)$ 
18:       for each  $skyline$  in  $Sky(R', p)$ 
19:          $dominate(skyline)$ 
20:       for each  $RI'$  in  $Sky(R', p)$ 
21:         If  $RI'[3] == 0$ 
22:            $RI'[3] = 1$ 
23:            $Q.insert(\{RI'[0], RI'[1] + estimate(p), RI'[2]\})$ 
24: return  $dominate(SkyS)$ 

```

The pseudo-code for the best solution was introduced previously. Algorithm2 is the pseudo-code for the approximate solution. The first line of $Sky(R', p)$ for the best solution at initialization represents how we dictate the need to consider the solution requests and the current point. $Sky(R')$, relaxes the constraint of domination and allows the same requests

to be solved. Line 13, the approximate solution differs from the optimal solution in that we are not searching for all the POIs that can solve at least one request. Instead, the becomes at least one request that can be solved and $dis(Route) + estimate(p)$ in the first k POI will only be considered for expansion. The above is a complete introduction to pseudo-code.

Algorithm 2 Greedy Skyline Search

Input: s (starting point), s_t (Start time), e (destination), R (the collection of requests), D (The collection of requests deadline)

Output: $Skys$

```

5:   $Skys = \{\}$ , Priority Queue  $Q = \{\}$ ,  $Sky(R') = \{\}$ 
6:   $Route = \langle s \rangle$ 
7:  Priority Queue  $Q.insert(\{Delay(Route), Dis(Route), Route\})$ 
8:  While  $Q \neq \{\}$ 
9:     $Route \leftarrow Q.pop()$ 
10:   If  $Route.R' == |R|$ 
11:      $Route' = Route + e$ 
12:      $Skys.insert(\{Delay(Route'), Dis(Route'), Route'\})$ 
13:     continue
14:    $POI_{List} = []$ 
15:   for  $POI\ p, p \in \cup_{r \in R} PSR(r), \forall 1 \leq i \leq k$  do
16:      $POI_{List}.insert(p)$ 
17:   for each  $p$  in  $POI_{List}[k]$  do
18:      $Route' = Route + p$ 
19:      $R' = R \cap \cup_{p \in Route'} RSP(p)$ 
20:      $Route_{information}\ RI = \{Delay(Route'), Dis(Route'), Route', 0\}$ 

```

```
21:   Sky(R').insert(RI)
22:   for each skyline in Sky(R')
23:     dominate(skyline)
24:   for each RI' in Sky(R')
25:     If RI'[3] == 0
26:       RI'[3] = 1
27:       Q.insert({RI'[0], RI'[1] + estimate(p), RI'[2]})
28: return dominate(SkyS)
```

Chapter 5 Experimental Evaluation

In the Experimental Evaluation section, we first describe the POI dataset used in the following experiments, which are divided into two main experiments. The first part compares different methods of adding the best solution, and the second part is a comparison between different methods of adding the approximate solution at different k values comparison. Each query in all experiments was averaged over 100 trials, with the starting point and destination of each trial randomly located at the query in the dataset area.

5.1 Data Collection and Setting

We use Google Places API to collect real-world POI data in Taiwan and Tainan. The area of the dataset is approximately 60×60 km². The dataset contains 10,208 POIs (N_p) and has 88 different types (N_s), and in the present for the study, we consider the type of services provided by a POI. A POI in the dataset has an average of 1.7 services and a maximum of 8 different services. Figure 5-1 shows the distribution of POIs in Tainan.

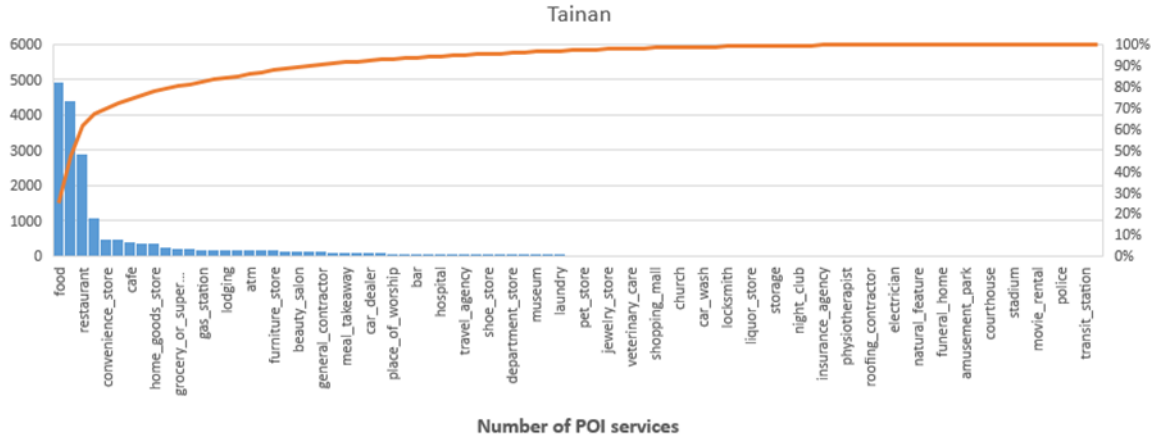


Figure 5-1 Number of POI Services

As well as the distribution of the number of POIs per service, it can be seen that most POIs are concentrated in the top 30% of all services, while one The maximum number of service POIs is 5,000. In the experiment, we randomly select services as user-specified requirements and use the number of service POIs as the basis for their selection. The

probability of the two POIs being under the principle of fairness. Each POI has its latitude and longitude, and the distance between two POI is measured in Euclidean distance. Table 5-1 introduces the parameters and default values we set in this experiment. N_R is the number of user queries. User day is the date of the user's query. Randomly select Monday to Sunday. User time is when a user queries, picking randomly from 24 hours in half-hour increments, e.g., 0 That's 12:00 am. 1 is 12:30 am. and so on.

Parameter	Description	Default	Value
N_R	Number of request	3	1 ~ 5
User Day	User query day	0~6	
User time	User query time	0~47	
User request deadline	User request deadline	30min or 1hr	

Table 5-1 parameters

5.2 Comparison of optimal algorithm

We have proposed the algorithm for the optimal solution for MRRP-TU. In this section, we will explore the comparison between different combinations of methods and different demand numbers. For optimal solution we propose three method Priority Queue P, Dominate Pruning D, and Estimate E. For approximate solution we relax the constraint of Skyline dominate into same service S.

5.2.1 Impact of methodology

Among the optimal solution, we propose the algorithms P, D, E. Based on the order of appearance of the three methods, we compare P, P+D, P +D+E three combinations in this section. Figure 5-2 represents the execution time for three different methodologies. Comparing P and P+D first, we can see that using the only P would take a lot of time because in order to find the optimal solution for MRRP-TU, we have to find all combinations of alignments. P+D because of dominate pruning since we can eliminate paths that will not be

a part of the optimal solution in the path expansion. We can see that P+D reduces the execution time by 96% over P. Therefore, we can determine that our proposed Dominate pruning strategy can significantly speed up MRRP-TU queries. Then we compare P+D with P+D+E. We find that the execution time of P+D+E is larger than that of P+D. In the P+D+E approach, it takes time to calculate the potential cost of the path, which is why P+D+E's average execution time is larger than that of P+D.

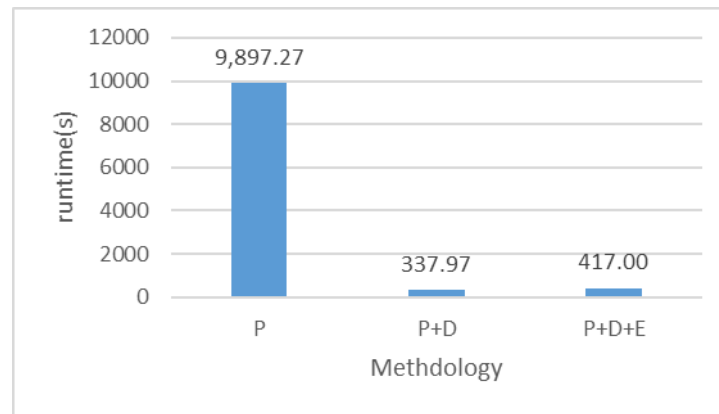


Figure 5-2 Impact of methodology

Although the average execution time of P+D+E is larger, because our algorithm is anytime algorithm, therefore, in addition to considering the algorithm's execution time to find the optimal solution, it is also necessary to look at the convergence Speed of the algorithm's. The figure is the quality convergence graph of P+D and P+D+E. At 0.3 seconds, the P+D+E algorithm does not output a solution because it needs to calculate the potential cost. However, at 0.1-second, P+D only finds 60% of the solution, while P+D+E finds 80% of the solution. If we look at the anytime algorithm, P+D+E can find a high-quality solution faster because it considers the potential cost.

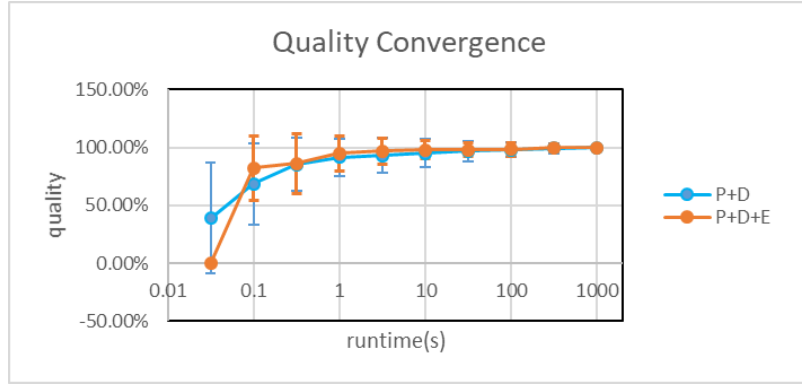


Figure 5-3 Convergence of methodology

Figure 5-4 shows the number of P+D and P+D+E expansions. We can see that P+D+E expands more paths than P+D. Therefore, it takes more time to find the best solution in the iteration process. The reason for the larger number of expansions is that we no longer order the priority queue by the number of requests solved after adding the potential cost. Therefore, the paths that expand first cannot dominate each other and instead increase the number of expansions.

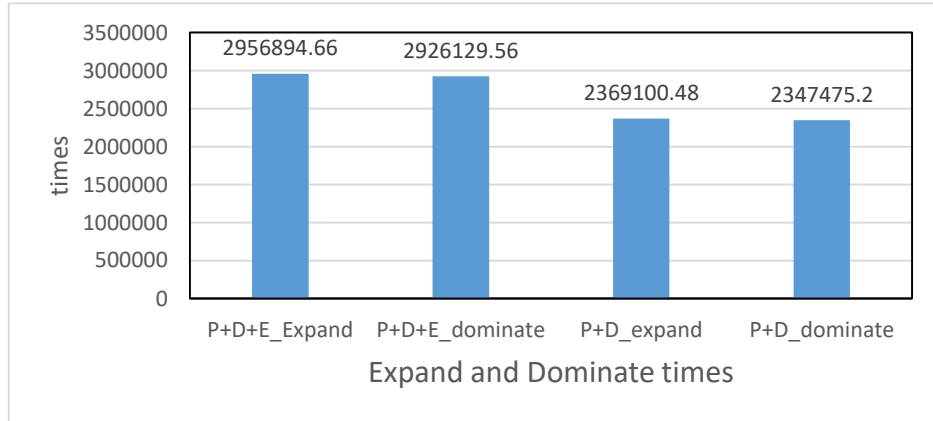


Figure 5-4 Number of Expand and Dominate times

5.2.2 Impact of Number of Request N_R

In this experiment, we test the algorithm under a different number of requirements, which ranges from 1 to 5. Figure 5-5 is the result of the experiment. We can see that the execution time to find the optimal solution increases exponentially as the requests increase. That is because the permutations increase significantly with each increase in the number of requests, and thus the execution time increases rapidly.

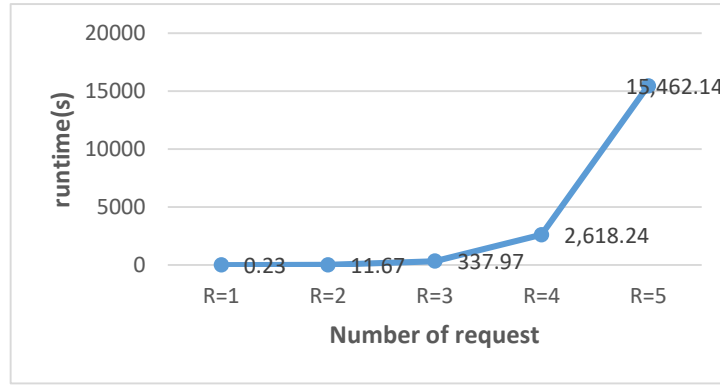


Figure 5-5 Impact of Number of Request

5.2.3 Impact of distance of Source and destination

In this experiment, we test the algorithm under different distance of source and distance, which from 0km to 30km. Fig is the result of the experiment. We can see that the execution time is lower when the distance between source and destination is smaller or larger. That is because the smaller or larger distance can eliminate more routes by dominate pruning strategy.

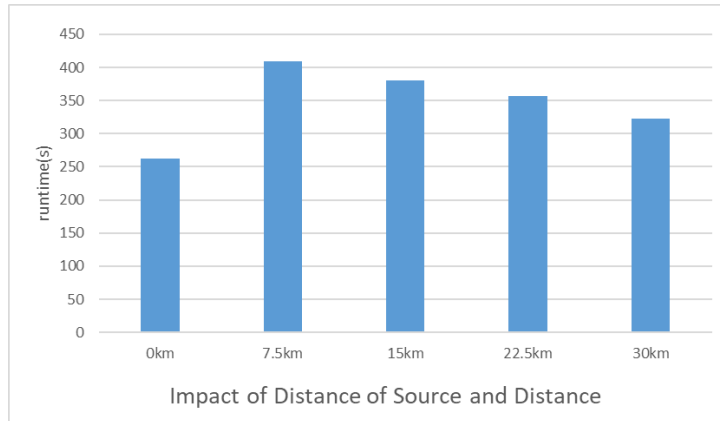


Figure 5-6 Impact of Distance of Source and Destination

5.2.4 Impact of Query Time

In this experiment, we test the algorithm under different query time, which from 8AM to 20PM. Figure 5-7 is the execution time of the experiment, Figure 5-8 is the numbers of POIs which is open in different time. Figure 5-9 is the numbers of expanding routes. We can see that there is no absolute relationship between the number of openings POI and the execution time. With Figure 5-9 we can see that if a better path is found, other routes are eliminated

before they are added to the candidate path and are not counted as expands, making execution faster.

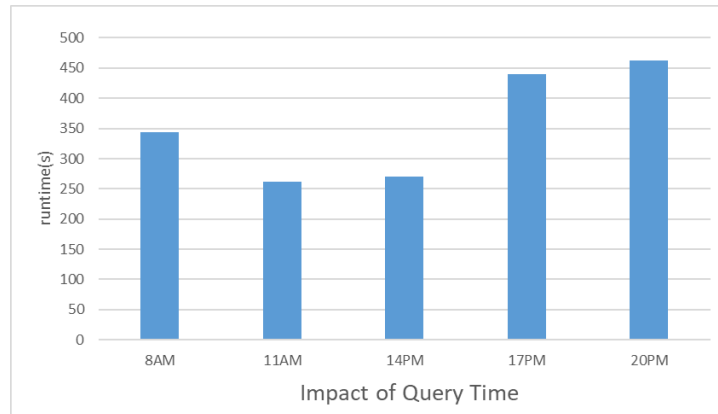


Figure 5-7 Impact of Query Time

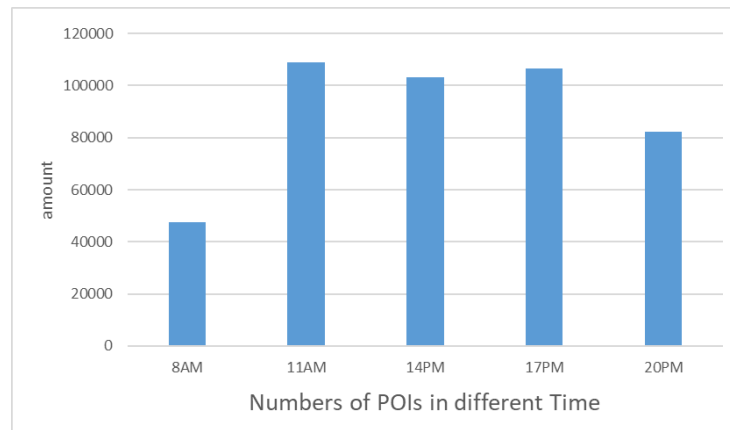


Figure 5-8 Numbers of POIs in different Time

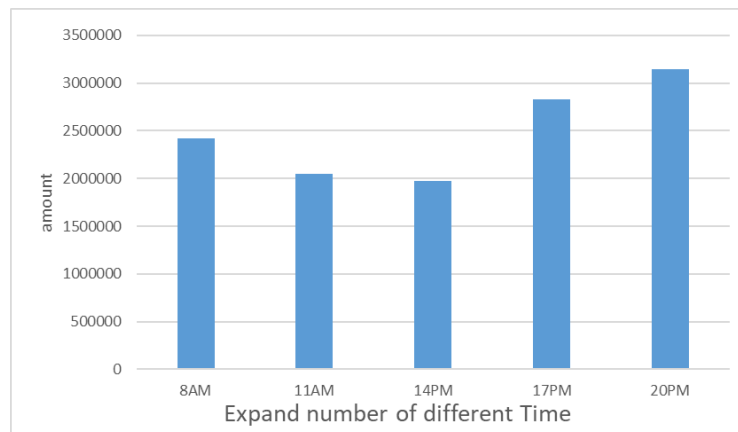


Figure 5-9 Expand number of different Time

5.3 Comparison of greedy algorithm

In this experiment, we refer to the way in which we relax the dominate pruning strategy as the Same services, S. Therefore, there will be a new method P+D+E+S. We discuss

approximate solution algorithms' effect on the quality of finding approximate solutions for different methods and different k-values. Figure 5-10 is the quality of the average path solution at different k-values and algorithms. We can see that since the difference between P and P+D is only an added pruning strategy and, therefore, the same in selecting the points, the average quality is also the same. The difference between P+D and P+D+E is that the k points selected for P+D+E are the POI after considering the potential cost. Therefore, the path solution's quality will be better than P+D, which does not take into account the potential cost. P+D+E+S is the worst performing path in $k = 20$ to 50 because we relax the constraint of the dominate pruning strategy, but it also makes the quality of this method of finding solutions decrease, but for $k = 100$ we can find that the quality of the P+D+E+S come to 96%.

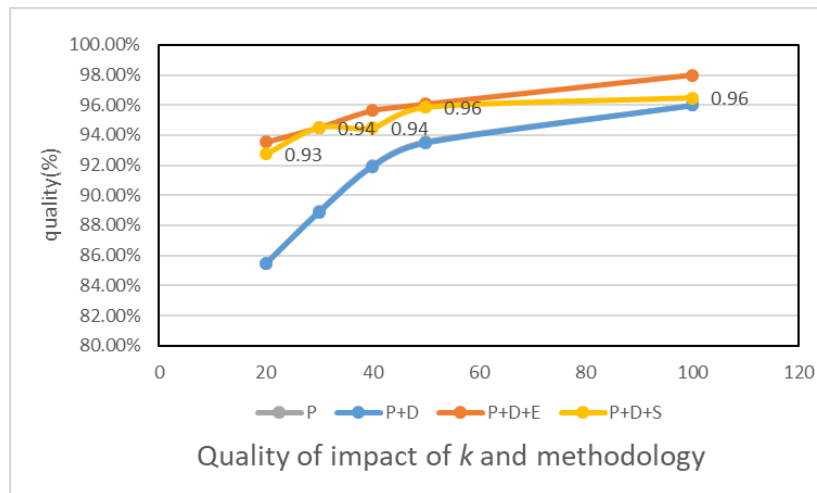


Figure 5-10 Quality of impact of k and methodology

Figure 5-11 is the average execution time for different values of k and the algorithm. We can see the same conclusion as in section 3.2.1. P takes the longest execution time; P+D+E is slightly higher than P+D. However, we can find that P+D+E+S finds 96% of the solutions with an average execution time of less than 1 second for $k = 100$.

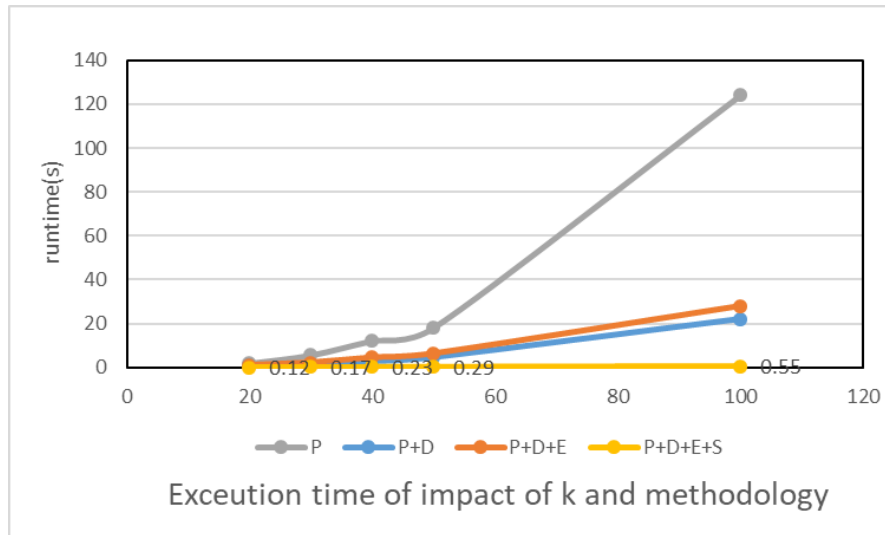


Figure 5-11 Execution time of impact of k and methodology

The approximation design's main purpose is to find a path that satisfies the user's requirements in a limited time. To further compare the differences between the different methods, we select the k-values that can be executed in 6 seconds from among the different methods to compare convergence graphs Figure 5-12.

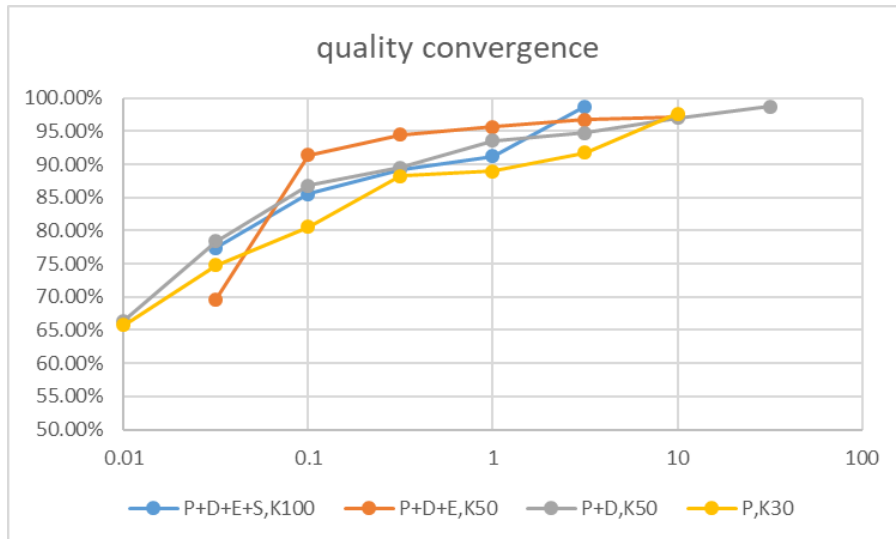


Figure 5-12 convergence of methodology

P, K=30; P+D, K=50; P+D+E, K=50; P+D+E+S, K=100, respectively. First, comparing P with P+D, we can see that P+D converges faster than P because P+D has a larger k-value than P. Therefore, the final solution's quality is better. Then comparing P+D with P+D+E, we

can also see that P+D+E converges faster than P+D after 0.1 P+D+E converges faster than P+D. Then we see P+D+E+S. We can see that it takes only 3 seconds to find a solution with 98% quality. In summary, we can find that P+D+E and P+D+E+S have their own advantages and disadvantages. P+D can be used if one wants to return a feasible solution quickly, and the quality of the feasible solution is still good. P+D+E+S can be used if one wants to find a better possible solution

5.4 Experimental Summary

In the experimental evaluation section, the main experiment divide into two parts, the first part is the comparison between different methods under the best solution and the second part is the comparison between different methods under different values of k for the approximate solution. The area is Taiwan-Tainan. Based on the three methods' order of appearance, we compare The execution time of the combination P, P+D, and P+D+E. We can see that P+D reduces the execution time by 96% compared to P, so we can be sure that the dominate pruning strategy can significantly speed up the MRRP-TU query. In the P+D+E approach, we need to calculate the potential cost of the path, which is why the average execution time of P+D+E larger than P+D. Because our algorithm is an anytime algorithm. In addition to the execution time of the algorithm to find the best solution, we also need to look at the convergence speed of the algorithm. On the other hand, P+D+E can find 80% of the solutions in 0.1 seconds. If we look at it from the point of view of anytime algorithm, P+D+E can find the High-quality solution faster because it takes into account the potential cost. We have tested the impact of the user's algorithm at different numbers of requests. We can see that the execution time to find the best solution increases exponentially as the number of requests increases.

In experiments with approximate solutions, we discuss the effect of approximate solution algorithms on different methods for finding approximate solution qualities at

different k -values, execution time, and the effect of convergence speed. P+D+E+S is the worst-performing algorithm for $k = 20$ to 50 , but for $k = 100$, we can found that P+D+E+S's quality reaches to 96%. And the average execution time is less than one second. In the comparison of convergence graph, We can see that P+D+E and P+D+E+S have their own advantages and disadvantages. P+D+E can be used if one wants to return a feasible solution quickly, and the quality still good, and P+D+E+S can be used if one wants to find a better feasible solution. In the experimental phase, we also showed that the skyline search algorithm has the ability to return the MRRP-TU Optimal solutions, approximate solutions, and the ability to return feasible solutions by using anytime algorithms.

Chapter 6 Conclusion and Future Work

This paper's main contribution is that we address a new problem called Multi-Request Route Planning with Time Urgency, MRRP-TU, and gives a basic definition of this problem. This question base on the user's requests and the POI's point information. Find a collection of skyline on a map with a path that satisfies all the user's requests. Compared to MRRP, MRRP-TU is more suitable for practical applications. Moreover, MRRP-TU for NP-hard problems. In this paper, we propose that the multi-objective search algorithm efficiently answers MRRP-TU queries. Since route queries require immediate response, we propose algorithms for approximate solutions in addition to the study of optimal solutions. In this paper, we propose an anytime algorithm that combines priority queues and pruning strategies to be able to find the Skyline collection paths before quickly returning sub-optimal solutions of a certain quality to answer MRRP-TU queries effectively.

We first solve the MRRP-TU query using brute force search, then we add the priority queue based on the fact that the brute force search queues do not have sorting capabilities. We find that most of the paths will not be part of the best solution for the search of all possibilities, so we propose the dominate pruning strategy filters out routes that we believe are unnecessary to continue expanding. The inclusion of a potential cost estimate allows the algorithm to find an 80% solution in 0.1 seconds. Finally, we propose conditions for kA^* and relaxation of the dominate pruning strategy. It gives our algorithm the ability to return high-quality approximate solutions in a limited time. The final skyline search algorithm returns the currently found sub-optimal solution in each iteration.

In the experimental part, we first evaluate our method with real-world data and combine different methods for comparing the best solution algorithm and approximate solution algorithm. The comparison includes the impacts of the number of requests, the impacts of

different methods, and the impacts of different k -values. The results show that in the best solution algorithm, our method P+D, P+D+E can significantly increase our queries' speed. In experiments with optimal solution convergence speed, our method also shows the ability to return solutions quickly. In experiments with approximate solutions, we also show us that the proposed method P+D+E+S can return high-quality solutions in a short time.

In future work, for data structure, we will consider different POI index structures including, R-tree, G-tree, Voronoi diagram, to store the data. To speed up the query, we will consider the Sort-Filter Skyline, Divide and conquer Skyline. we plan to improve the current algorithm further among the optimal solution algorithms. We will try to propose a method to estimate the delay time. Since our priority queue is to consider the delay time first, then consider the road distance, so we assume that if there is a way to estimate delays, we can find a better solution in a shorter time. In the approximate solution algorithm, we will try to select k POIs that have more potential to approach the best solution for expansion. Final we want to change this question from a single query to a group query so that the skyline query can become a Pareto optimality question. Extensive experiments will verify the validity and magnitude of the improvement. We will collect more real-world data of different scales to evaluate the experiments' efficacy more completely.

References

- [1] C. S. Jensen, J. Kolářvr, T. B. Pedersen and I. Timko, "Nearest Neighbor Queries in Road Networks," *In ACM GIS*, pp. 1-8, Nov. 2003.
- [2] E. W. Dijkstra, "A Note on Two Problems in Connection with Graphs," *Numerische Mathematik*, 1(1): 269-271, 1959.
- [3] P. E. Hart, N. J. Nilsson and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Trans. on Systems Science and Cybernetics*, 4(2): 100-107, Feb. 2007.
- [4] K. Menger, "Das Botenproblem," *Ergebnisse Eines Mathematischen Kolloquiums*, 2: 11-12, 1932.
- [5] M. Albayraka and N. Allahverdiyev, "Development A New Mutation Operator to Solve the Traveling Salesman Problem by Aid of Genetic Algorithms," *Expert Systems with Applications*, 38(3): 1313-1320, Mar. 2011.
- [6] J. Dai, B. Yang, C. Guo and Z. Ding, "Personalized Route Recommendation using Big Trajectory Data," *In IEEE ICDE*, pp. 543-554, Apr. 2015.
- [7] F. Li, D. Cheng, M. Hadjieleftheriou, G. Kollios and S.-H. Teng, "On Trip Planning Queries in Spatial Databases," *In SSTD*, pp. 273-290, Aug. 2005.
- [8] M. Sharifzadeh, M. Kolahdouzan and C. Shahabi, "The Optimal Sequenced Route Query," *VLDB Journal*, 17(4): 765-787, Jul. 2008.
- [9] H. Chen, W. S. Ku, M. T. Sun and R. Zimmermann, "The Multi-Rule Partial Sequenced Route Query," *In ACM GIS*, Article No. 10. Nov. 2008.
- [10] H. Liu, C. Jin, B. Yang and A. Zhou, "Finding Top-k Optimal Sequenced Routes," *In IEEE ICDE*, pp. 569-580, Feb. 2018.
- [11] E. H. C. Lu, H. S. Chen and V. S. Tseng, "An Efficient Framework for Multirequest

- Route Planning in Urban Environments," *IEEE Trans. on Intelligent Transportation Systems*, 18(4): 869-879, Apr. 2017.
- [12] Z. Song and N. Roussopoulos, "K-Nearest Neighbor Search for Moving Query Point," *In SSTD*, pp. 79-96, Jul. 2001.
- [13] Y. Tao, D. Papadias and Q. Shen, "Continuous Nearest Neighbor Search," *In VLDB*, pp. 287-298, Aug. 2002.
- [14] E. H.-C. Lu, C.-C. Lin and V. S. Tseng, "Mining The Shortest Path within a Travel Time Constraint in Road Network Environments," *In IEEE ITSC*, pp. 593-598, Oct. 2008.
- [15] C.-C Lee, Y.-H. Wu and A. L. P. Chen. "Continuous Evaluation of Fastest Path Queries on Road Networks," *In SSTD*, pp. 20-37, Jul. 2007.
- [16] E. H.-C. Lu, C.-W. Huang and V. S. Tseng, "Continuous Fastest Path Planning in Road Networks by Mining Real-Time Traffic Event Information," *In ISII*, pp. 969-974, Sept. 2009.
- [17] J. Xu, Y. Gao, C. Liu, L. Zhao and Z. Ding, "Efficient Route Search on Hierarchical Dynamic Road Network," *Distributed and Parallel Databases*, 33: 227-252, Mar. 2014.
- [18] I. Hefez, Y. Kanza and R. Levin, "Tarsius: A System for Traffic-aware Route Search under Conditions of Uncertainty," *In ACM GIS*, pp. 517-520, Nov. 2011.
- [19] U. Demiryurek, F. Banaei-Kashani, C. Shahabi and A. Ranganathan, "Online Computation of Fastest Path in Time-Dependent Spatial Networks," *In SSTD*, pp. 92-111, Aug. 2011.
- [20] X. Ma, S. Shekhar and H. Xiong, "Multi-Type Nearest Neighbor Queries in Road Networks with Time Window Constraints," *In ACM GIS*, pp. 484-487, Nov. 2009.
- [21] X. Cao, L. Chen, G. Cong, J. Guan, N.-T. Phan and X. Xiao, "KORS: Keyword-aware Optimal Route Search System," *In IEEE ICDE*, pp. 1340-1343, Apr. 2013.

- [22] C. F. Costa, M. A. Nascimento, J. A. Macêdo, Y. Theodoridis, N. Pelekis and J. Machado, "Optimal Time-Dependent Sequenced Route Queries in Road Networks," *In ACM GIS*, Article No. 56, Nov. 2015.
- [23] Y. Huang, B. H. Lin and V. S. Tseng, "Efficient Multi-Destinations Route Planning with Deadlines and Cost Constraints," *In IEEE MDM*, pp. 228-233, May 2017.
- [24] F. Luan, Q. Li, K. Cao and L. Wang, "Multi-Constrained Dominate Route Queries in Time-Dependent Road Networks," *In Big Data*, pp. 135-148, Oct. 2018.
- [25] J. Li, J. Hu, V. Engel, C. Zong and X. Xia, "An Efficient Multi-request Route Planning Framework Based on Grid Index and Heuristic Function," *In ADMA*, pp. 737-749, Nov. 2019.
- [26] S. Borzsonyi, D. Kossmann and K. Stocker, "The Skyline Operator," *In IEEE ICDE*, pp. 421-430, Apr. 2001.
- [27] X. Huang and C. S. Jensen, "In-Route Skyline Querying for Location-Based Services," *In W2GIS*, pp. 120-135, Nov. 2004.
- [28] M. Sharifzadeh and C. Shahabi, "The Spatial Skyline Queries," *In VLDB*, pp. 751-762, Sept. 2006.
- [29] Z. Huang, H. Lu, B. C. Ooi and A. Tung, "Continuous Skyline Queries for Moving Objects," *IEEE Trans. on Knowledge and Data Engineering*, 18(12): 1645-1658, Dec. 2006.
- [30] K. Deng, X. Zhou and H. T. Shen, "Multi-Source Skyline Query Processing in Road Networks," *In IEEE ICDE*, pp. 796-805, Apr. 2007.
- [31] B. Zheng, K. C. K. Lee, W.-C. Lee, "Location-Dependent Skyline Query," *In IEEE MDM*, pp. 148-155, Apr. 2008.
- [32] H. P. Kriegel, M. Renz and M. Schubert, "Route Skyline Queries: A Multi-Preference Path Planning Approach," *In IEEE ICDE*, pp. 261-272, Mar. 2010.

- [33] Y. K. Huang, C. H. Chang and C. Lee, "Continuous Distance-Based Skyline Queries in Road Networks," *Information Systems*, 37(7): 611-633, Nov. 2012.
- [34] B. Yang, C. Guo, C. S. Jensen, M. Kaul and S. Shang, "Stochastic Skyline Route Planning under Time-Varying Uncertainty," *In IEEE ICDE*, pp. 136-147, Apr. 2014.
- [35] M. Shekelyan, G. Josse and M. Schubert, "Linear Path Skylines in Multicriteria Networks," *In IEEE ICDE*, pp. 459-470, Apr. 2015.
- [36] Q. Gong, H. Cao and P. Nagarkar, "Skyline Queries Constrained by Multi-cost Transportation Networks," *In IEEE ICDE*, pp. 926-937, Apr. 2019.
- [37] P. Yawalkar and S. Ranu, "Route Recommendations on Road Networks for Arbitrary User Preference Functions," *In IEEE ICDE*, pp. 602-613, Apr. 2019.